



۸۱۰۱۰۱۴۰۸

۸۱۰۱۰۱۳۹۲

مبانی رایانش توزیع شده

نام و نام خانوادگی اعضای گروه:

بابک حسینی محتشم

بهراد بینایی



تمرین شماره ۲

۱ مقدمه

در این تمرین با استفاده از مفهوم RPC و به طور خاص با کمک پروتکل JSON-RPC ابتدا به پیاده سازی یک سیستم توزیع شده شامل سه سرویس وب سرور، سرور احراز هویت و سرور فایل می پردازیم. در نهایت با ایجاد تغییرات در وب سرور یک سیستم شامل Publisher و Subscriber ایجاد می کنیم که در صورت استفاده بیش از حد وب سرور از حافظه، Subscriber اعلانی به کاربر می دهد.

۲ معماری کلی سیستم

سیستم پیاده سازی شده از چهار سرویس زیر تشکیل می شود:

۱. **سرور احراز هویت (Auth Server):** سرویسی که روی پورت 8082 گوش می دهد و عملیات ورود کاربران را از طریق JSON-RPC over TCP مدیریت می کند.
۲. **سرور فایل (File Server):** سرویسی که روی پورت 8083 گوش می دهد و فایل ها را از طریق JSON-RPC over TCP در اختیار کلاینت ها قرار می دهد.
۳. **سرور وب / ناشر (Web Server / Publisher):** سرویس اصلی روی پورت 8081 که رابط کاربری وب را ارائه می دهد، با دو سرویس بالا ارتباط برقرار می کند، و رویدادهای مصرف حافظه را منتشر می کند.
۴. **مشترک (Subscriber):** سرویسی روی پورت 8084 که رویدادهای هشدار مصرف حافظه را از طریق HTTP POST دریافت و نمایش می دهد.

در این پروتکل از JSON-RPC روی TCP استفاده شده است. دلایل اصلی این انتخاب عبارتند از:

- **سادگی پیاده سازی:** Go به صورت پیش فرض بسته های net/rpc و net/rpc/jsonrpc را پشتیبانی می کند. با چند خط کد می توان یک سرویس RPC کامل راه اندازی کرد.

□ **خوانایی پیام‌ها:** فرمت JSON انسانی‌خوان است و اشکال‌زدایی و لاگ‌کردن درخواست‌ها را بسیار آسان می‌کند. برای مثال می‌توان با netcat مستقیماً درخواست فرستاد و پاسخ را دید.

□ **عدم نیاز به IDL:** برخلاف gRPC که نیاز به فایل proto. و تولید کد دارد، JSON-RPC بدون مرحله اضافی کار می‌کند و برای یک تمرین آموزشی با مقیاس کوچک مناسب است.

□ **سربرار ارتباطی کمتر نسبت به REST:** REST معمولاً روی HTTP/1.1 سوار می‌شود و هدرهای زیادی دارد. JSON-RPC over TCP هدر اضافی ندارد و برای ارتباط بین سرویس‌ها سبک‌تر است.

□ **مناسب برای ارتباطات داخلی:** در این تمرین، سرویس‌ها درون شبکه خصوصی ماشین‌های مجازی با هم صحبت می‌کنند. نیازی به ویژگی‌های REST مانند caching یا statelessness نداریم؛ فراخوانی مستقیم تابع راه دور همان چیزی است که نیاز داریم.

اگرچه gRPC کارایی بالاتری دارد و برای سیستم‌های بزرگ صنعتی مناسب‌تر است، اما پیچیدگی اضافی آن برای اهداف این تمرین توجیه‌پذیر نبود. همچنین REST با وجود سادگی در معماری‌های عمومی، برای ارتباط همزمان بین سرویس‌های اختصاصی، حجم پیام بیشتری ارسال می‌کند و نگهداری اندپوینت‌های متعدد در مقایسه با یک متد RPC ساده، دشوارتر است.

۳ سرور احراز هویت (VM2)

۱.۳ ساختار و وظایف

سرور احراز هویت یک متد RPC به نام AuthService.Login را در اختیار کلاینت‌ها می‌گذارد. این متد یک جفت نام کاربری و رمز عبور دریافت می‌کند و نتیجه بررسی صحت آن‌ها را بازمی‌گرداند. اطلاعات کاربران در فایل users.json ذخیره می‌شود. هر رکورد کاربری شامل نام کاربری و هش SHA-256 رمز عبور است. در زمان راه‌اندازی سرور، این فایل خوانده شده و در یک map در حافظه نگهداری می‌شود تا جستجوی سریع فراهم باشد:

```
type User struct {
    Username    string `json:"username"`
    PasswordHash string `json:"password-hash"`
}
```

هنگام ورود، سرور رمز عبور دریافت‌شده را با SHA-256 هش می‌کند و با هش ذخیره‌شده مقایسه می‌کند. اگر کاربر وجود نداشته باشد یا هش‌ها مطابقت نداشته باشند، پیام خطا برگشت داده می‌شود:

```

storedHash, exists := a.users[args.Username]
hashBytes := sha256.Sum256([]byte(args.Password))
passHash := hex.EncodeToString(hashBytes[:])

if !exists || storedHash != passHash {
    reply.Success = false
    reply.Error = "Invalid username or password"
    return nil
}
reply.Success = true
reply.Error = ""

```

سرور با استفاده از `net.Listen("tcp", addr)` روی پورت مشخص شده گوش می‌دهد. به ازای هر اتصال ورودی، یک goroutine جدید ایجاد شده و `jsonrpc.ServeConn(conn)` فراخوانی می‌شود. همچنین سیگنال‌های `SIGINT` و `SIGTERM` برای خاموشی نرم مدیریت می‌شوند.

۴ سرور فایل (VM3)

۱.۴ ساختار و وظایف

سرور فایل یک متد `RPC` به نام `FileService.GetFile` ارائه می‌دهد که نام فایل را دریافت کرده و محتوای آن را به صورت `base64` رمزگذاری شده بازمی‌گرداند. پاسخ شامل محتوا، نوع محتوا (`MIME type`)، و اندازه فایل است. نوع محتوا از پسوند فایل تشخیص داده می‌شود و در `MIME type` پاسخ قرار می‌گیرد تا کلاینت بداند چگونه با محتوا رفتار کند. در پوشه `files/images/` فایل `sample.png` به عنوان نمونه قرار داده شده است.

۵ سرور وب (VM1) □ نسخه پایه (بدون Pub/Sub)

۱.۵ ساختار کلی

سرور وب یک وب‌سرور ساده است که صفحات `HTML` را ارائه می‌دهد. مسیرهای زیر پیاده‌سازی شده‌اند:

□ / □ نمایش فرم ورود (`GET`) و پردازش ورود (`POST`)

□ /static/ □ ارائه فایل‌های استاتیک (مانند `style.css`)

قالب login.html در پوشه templates قرار دارد و از فایل static/style.css برای ظاهر استفاده می‌کند. پس از دریافت اطلاعات ورود از فرم، سرور وب به‌عنوان کلاینت RPC به سرور احراز هویت متصل می‌شود. در صورت موفقیت، بلافاصله به سرور فایل متصل شده و فایل درخواستی را دریافت و مستقیم به مرورگر کاربر ارسال می‌کند. هر RPC call یک اتصال TCP جدید باز می‌کند، عملیات را انجام می‌دهد، و اتصال را می‌بندد. این رویکرد ساده است اما در بار بالا هزینه‌بر خواهد بود.

۶ بخش سوم: Pub/Sub و پایش مصرف حافظه

۱.۶ معماری ناشر و مشترک

در این بخش، سرور وب به یک publisher تبدیل می‌شود که علاوه بر قابلیت‌های قبلی، با هر بار اشغال حافظه مصرف حافظه خود را بررسی کرده و در صورت تجاوز از آستانه، یک رویداد HIGH_MEMORY_USAGE را به subscriber ارسال می‌کند. مشترک یک سرور HTTP ساده است که رویدادها را دریافت و در خروجی استاندارد چاپ می‌کند.

برای اندازه‌گیری حافظه از بسته runtime و ساختار MemStats استفاده شده است. فیلد Sys میزان حافظه کل تخصیص داده شده توسط سیستم را نشان می‌دهد که برای این تمرین مناسب است.

```
func getMemoryUsageMB() uint64 {
    var stats runtime.MemStats
    runtime.ReadMemStats(&stats)
    return stats.Sys / (1024 * 1024)
}
```

یک goroutine هر ۱۰ ثانیه یک بار این تابع را فراخوانی کرده و در صورت عبور از آستانه (پیش‌فرض ۳۰۰ مگابایت)، یک رویداد ایجاد و به مشترک ارسال می‌کند.

اندپوینت consume-memory/ با پارامتر mb مقدار مشخصی حافظه (بر حسب مگابایت) تخصیص می‌دهد و آن را در یک متغیر سراسری storedMem نگهداری می‌کند تا توسط آزاد نشود. بلافاصله پس از تخصیص، مصرف حافظه بررسی شده و در صورت لزوم رویداد ارسال می‌شود.

```
data := make([]byte, mb*1024*1024)
for i := range data {
    data[i] = byte(i % 256)
}
storedMem = append(storedMem, data)
```

۲.۶ پیاده‌سازی مشترک (subscriber)

مشترک یک HTTP server ساده روی پورت ۸۰۸۴ است که مسیر /event را برای درخواست‌های POST ارائه می‌دهد. بدنه درخواست به ساختار MemoryEvent تبدیل شده و جزئیات هشدار به صورت منظم چاپ می‌شود:

```
fmt.Printf("\n[ALERT] High memory usage detected!\n")
fmt.Printf("  Service:      %s\n", event.Service)
fmt.Printf("  Event type:    %s\n", event.EventType)
fmt.Printf("  Memory:       %d MB (threshold %d MB)\n", event.MemoryMB, event.ThresholdMB)
fmt.Printf("  Timestamp:    %s\n", event.Timestamp)
```

۷ راه‌اندازی ماشین‌های مجازی

برای پیاده‌سازی سیستم توزیع‌شده، از سه ماشین مجازی مبتنی بر VirtualBox استفاده شد. به منظور تسریع در راه‌اندازی، ابتدا یک ماشین پایه ایجاد شده و سپس با استفاده از قابلیت Clone در VirtualBox، دو نسخه دیگر از آن تهیه شد.

در فرآیند کلون کردن، برای هر ماشین مجازی تنظیمات شبکه به صورت مجزا پیکربندی شد تا هر یک دارای MAC Address و آدرس IP منحصر به فرد باشد. همچنین نوع اتصال شبکه (Network Adapter) به گونه‌ای تنظیم شد که ماشین‌ها بتوانند در یک شبکه مشترک با یکدیگر ارتباط برقرار کنند. پس از اجرای ماشین‌ها، آدرس IP هر یک با استفاده از دستور زیر به دست آمد:

```
ifconfig
```

۸ ارزیابی

مراحل تست مطابق خواسته تمرین به ترتیب زیر انجام شد. اسکرین‌شات‌های مربوطه در پوشه screenshots موجود است.

۱. آدرس ماشین‌های مجازی:

VM1: 192.168.56.101:8081 □

VM2: 192.168.56.103:8082 □

VM3: 192.168.56.102:8083 □

```

babak@babak-VirtualBox: ~/Code/GoLang/2026/DS/HW2/pubsub
68.56.102:8083 -aserver 192.168.56.103:8082
2026/05/25 09:51:58 Web server listening on 0.0.0.0:8081
2026/05/25 09:51:58 Auth server RPC at 192.168.56.103:8082
2026/05/25 09:51:58 File server RPC at 192.168.56.102:8083
2026/05/25 09:51:58 Will display file: images/sample.png after login
2026/05/25 09:51:58 Subscriber endpoint: http://localhost:8084/event

babak@babak-VirtualBox: ~/Code/GoLang/2026/DS/HW2/pubsub
flags=73<UP,LOOPBACK,RUNNING> mtu 65536
inet 127.0.0.1 netmask 255.0.0.0
inet6 ::1 prefixlen 128 scopeid 0x10<host>
loop txqueuelen 1000 (Local Loopback)
RX packets 5334 bytes 382644 (382.6 KB)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 5334 bytes 382644 (382.6 KB)
TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

virbr0: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
inet 192.168.122.1 netmask 255.255.255.0 broadcast 192.168.122.255
ether 52:54:00:3b:c0:ea txqueuelen 1000 (Ethernet)
RX packets 0 bytes 0 (0.0 B)
RX errors 0 dropped 0 overruns 0 frame 0
TX packets 0 bytes 0 (0.0 B)
TX errors 0 dropped 41 overruns 0 carrier 0 collisions 0

babak@babak-VirtualBox: ~/Code/GoLang/2026/DS/HW2/pubsub$ go run subscriber.go
2026/05/25 09:52:26 Subscriber listening on 0.0.0.0:8084

babak@babak-VirtualBox: ~/Code/GoLang/2026/DS/HW2/file-vn$ go run main.go
2026/05/25 09:48:05 File server listening on 0.0.0.0:8083 (JSON-RPC over TCP)

babak@babak-VirtualBox: ~/Code/GoLang/2026/DS/HW2/auth-vn$ go run main.go
2026/05/25 09:48:50 Loaded 4 users from users.json
2026/05/25 09:48:50 Auth server listening on 0.0.0.0:8082 (JSON-RPC over TCP)

```

شکل ۱: اجرای همزمان ماشین‌های مجازی و سرویس‌ها

Subscriber: 192.168.56.101:8084 (می‌تواند روی هر ماشین‌ی اجرا شود)

۲. اجرای سرور احراز هویت:

```
$ cd auth-vm
$ go run main.go -port 8082 -users users.json
```

۳. اجرای سرور فایل:

```
$ cd file-vm
$ go run main.go -port 8083 -dir ./files
```

۴. اجرای ناشر (publisher):

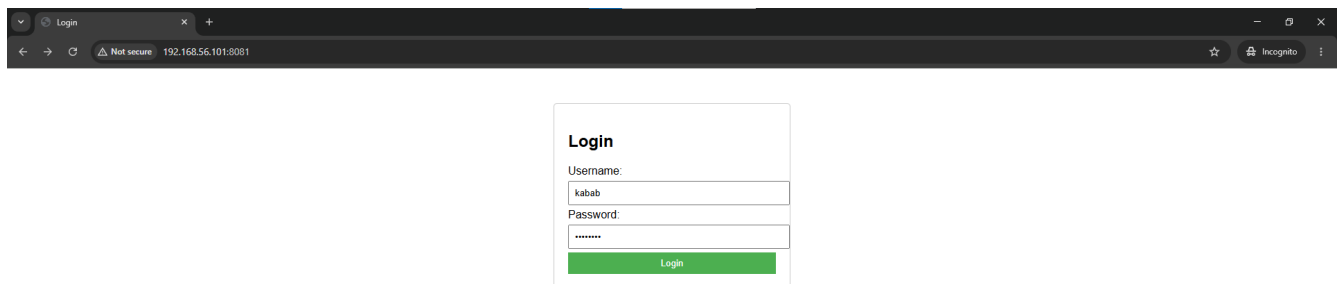
```
$ cd pubsub
$ go run publisher.go -port 8081 \
  -aserver 103:8082.56.168.192 \
  -fserver 102:8083.56.168.192 \
```

```
-file images/sample.png \  
-subscriber http://192.101:8084.56.168/event \  
-threshold 300
```

۵. اجرای مشترک:

```
$ cd pubsub  
$ go run subscriber.go -port 8084
```

۶. باز کردن صفحه وب از مرورگر میزبان: `http://192.168.56.101:8081`



شکل ۲: مرورگر در حال نمایش صفحه ورود

۷. تلاش با اطلاعات نادرست: `username: alice, password: wrong` ❌ پیام خطای `Login failed: Invalid username or password`

۸. تلاش با اطلاعات صحیح: `username: alice, password: alice123` ✅ صفحه موفقیت همراه با تصویر `sample.png` نمایش داده شد.

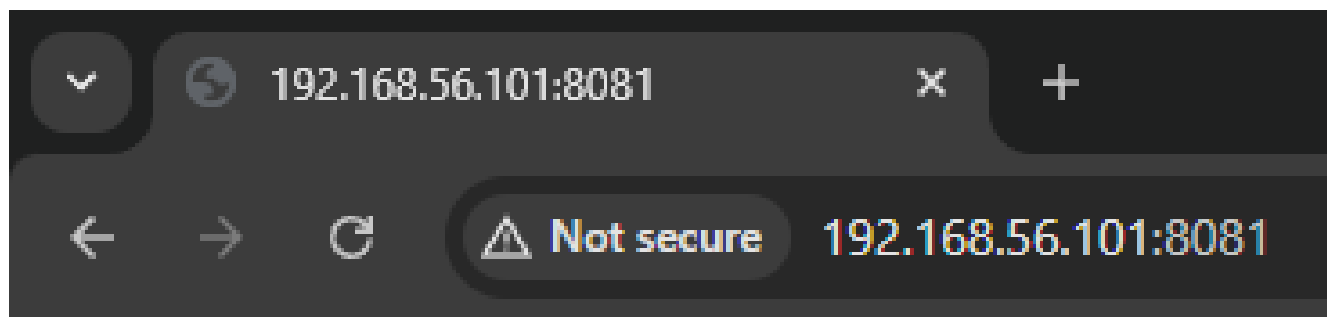
۹. دریافت فایل از VM3 از طریق RPC در لاگ سرور فایل قابل مشاهده است:

۱۰. افزایش عمدی مصرف حافظه با فراخوانی مکرر:

```
curl "http://192.101:8081.56.168/consume-memory?mb=100"
```

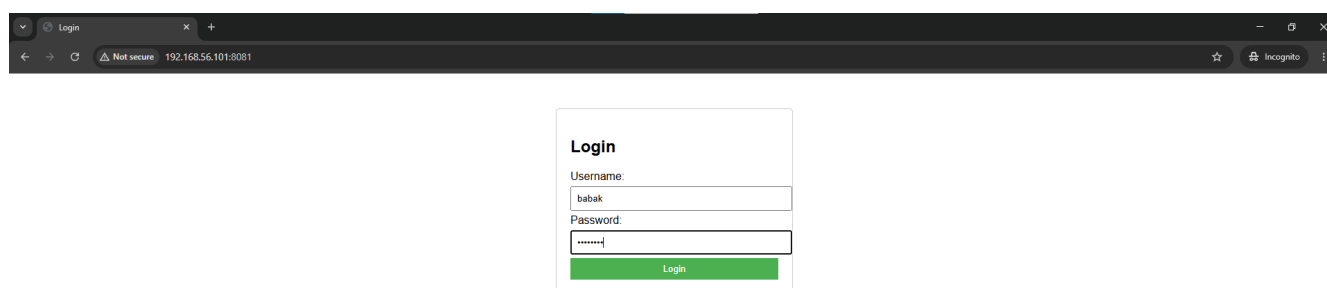
پس از سه بار فراخوانی (۳۰۰ مگابایت)، آستانه رد می شود و رویداد ارسال می گردد.

۱۱. مشاهده هشدار در مشترک:



Login failed: Invalid username or password

شکل ۳: نمایش خطای ورود ناموفق



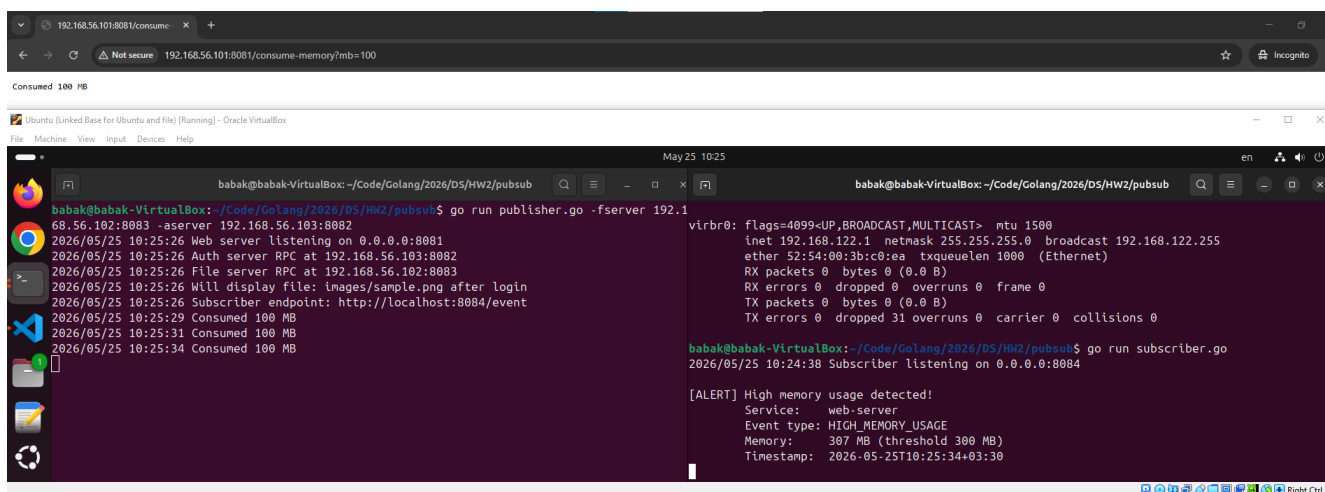
شکل ۴: نمایش فایل پس از ورود موفق

۹ پیکربندی از طریق آرگومان‌های خط فرمان

تمام سرویس‌ها از flag استاندارد Go برای پیکربندی استفاده می‌کنند. جدول زیر پارامترهای قابل تنظیم هر سرویس را نشان می‌دهد:



شکل ۵: درخواست RPC دریافت فایل در لاگ سرور فایل



شکل ۶: خروجی مشترک پس از دریافت رویداد هشدار

سرویس	پارامتر	مقدار پیش فرض
Auth Server	-port	8082
	-users	users.json
File Server	-port	8083
	-dir	./files
Web Server	-port	8081
	-aserver	localhost:8082
	-fserver	localhost:8083
	-file	images/sample.png
	-subscriber	http://localhost:8084/event
	-threshold	300 (مگابایت)

۱۰ مشکلات و محدودیت‌های پیاده‌سازی

- امنیت رمز عبور: از SHA-256 بدون salt استفاده شده است که در برابر حملات آسیب‌پذیر است. برای بهبود، می‌توان از bcrypt و salt تصادفی استفاده کرد.
- عدم استفاده از TLS: ارتباطات RPC و HTTP رمزنگاری نشده‌اند. در محیط واقعی باید از TLS استفاده شود.
- مدیریت اتصالات: هر RPC call یک اتصال TCP جدید ایجاد می‌کند که برای بار بالا بهینه نیست. می‌توان از connection pooling استفاده کرد.
- خطاپذیری: در صورت عدم دسترسی به سرور احراز هویت یا فایل، پیام خطای مناسبی به کاربر نمایش داده می‌شود اما مکانیزم تلاش مجدد (retry) پیاده‌سازی نشده است.

۱۱ نتیجه‌گیری

تمامی بخش‌های خواسته شده در تمرین شامل راه‌اندازی سه ماشین مجازی مستقل، ارتباط از طریق JSON-RPC، جداسازی سرویس وب، احراز هویت و فایل، پیاده‌سازی Pub/Sub ساده با پایش مصرف حافظه و ارائه روش افزایش عمده حافظه، با موفقیت پیاده‌سازی و تست شد. گزارش به همراه اسکرین‌شات‌های اجرا (در پوشه screenshots) و README‌های مجزا برای هر سرویس ارائه گردیده است.